Memoria 1: Fundamentos, versionamiento y tracking

Integrante: Jeferson Torrado Bayona
Repositorio: https://github.com/JefersonTB/proyecto-mlops-2026-II

1. Objetivo

Preparar la base reproducible de un proyecto de MLOps mediante control de versiones, gestión de dependencias y validación de la integración entre Prefect y MLflow. Esta primera actividad establece el entorno para el trabajo posterior del curso.

2. Descripción del trabajo

Se creó un repositorio en GitHub y se clonó en Windows. Se prepararon las carpetas src, flows, configs, data, tests y docs. Se configuró un entorno virtual local con Poetry y Python 3.13.

Se instalaron dependencias con versiones fijadas y se generó poetry.lock. Se excluyeron del control de versiones el entorno virtual y los archivos locales de MLflow.

Se iniciaron MLflow y Prefect en terminales separadas. Se descargó del repositorio del curso el archivo flows/prueba_flow.py y se ejecutó con las variables de conexión a ambos servidores.

El flow de validación calcula la suma de los números del 1 al 5 y registra el resultado en MLflow. No se implementó una pipeline propia de entrenamiento.

3. Decisiones técnicas

Se utilizó Poetry para gestionar el entorno y conservar las versiones resueltas en un archivo lock. Frente a instalar paquetes globalmente con pip, permite aislar las dependencias del proyecto.

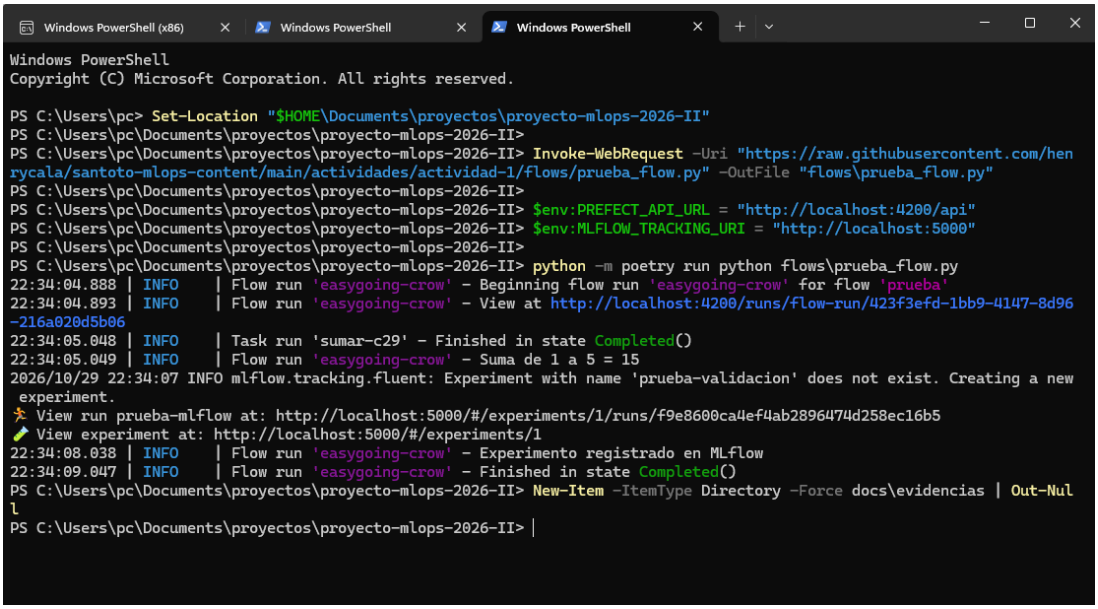
Se eligió la ejecución local sin Docker.

Se utilizó Python 3.13, ya instalado y dentro del rango admitido por la actividad. Se restringió el proyecto a esa versión menor para reducir diferencias entre entornos.

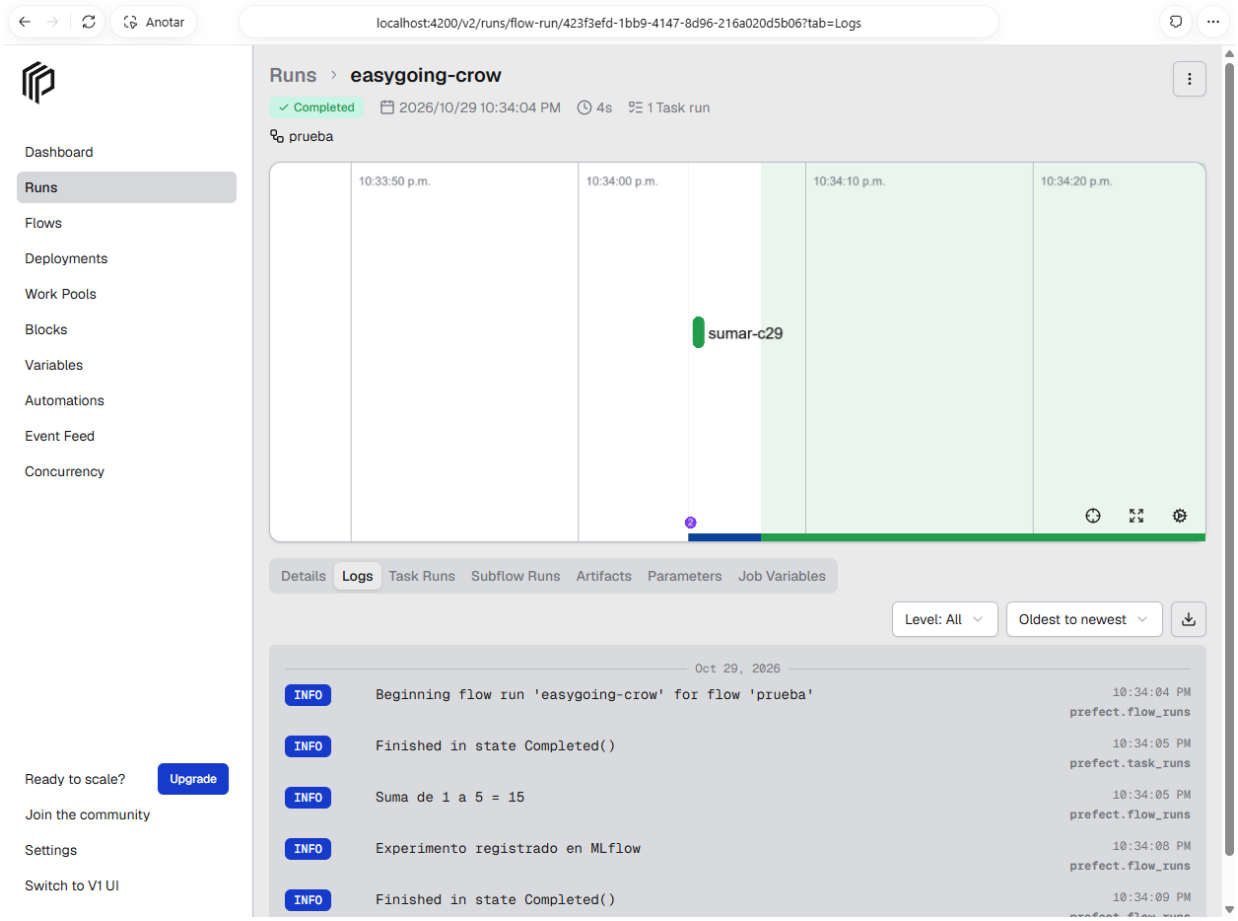
4. Evidencias

Las siguientes imágenes corresponden al desarrollo de la actividad:

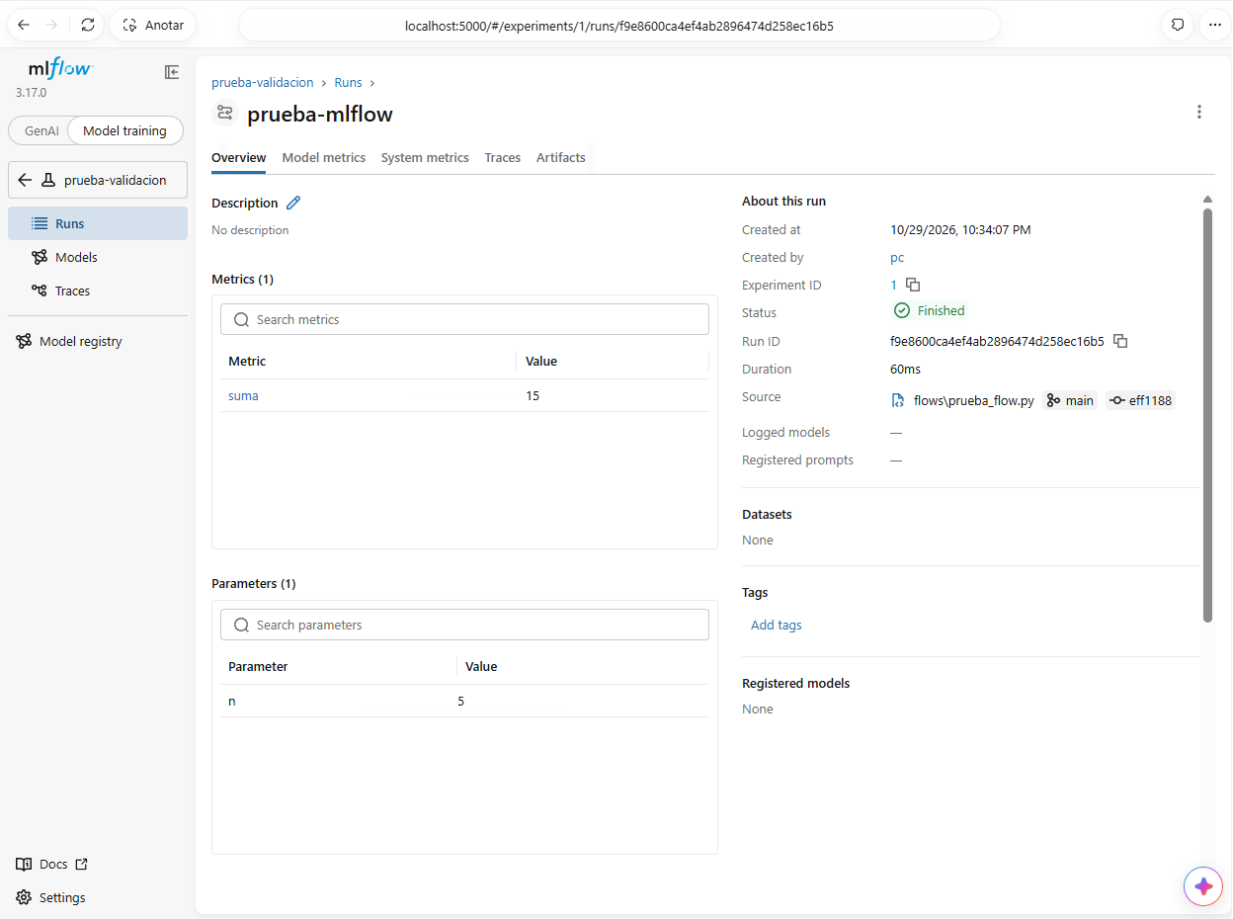
Ejecución en terminal



Run de Prefect



Run de MLflow



El código del flow proviene del repositorio del curso, no se presenta como desarrollo original del entregable.

5. Resultados

- Las importaciones de Prefect, MLflow y scikit-learn finalizaron correctamente.
- Las interfaces locales de MLflow y Prefect cargaron.
- El flow prueba, con nombre de ejecución easygoing-crow, terminó en Completed.
- El log mostró: Suma de 1 a 5 = 15.
- La terminal confirmó el registro del experimento en MLflow.
- poetry install terminó sin errores usando el lock existente.

La suma es un resultado de validación técnica, no una métrica de desempeño de un modelo predictivo.

6. Problemas encontrados

PowerShell no reconocía Poetry inicialmente. Se instaló con pip y se utilizó python -m poetry para invocarlo.

El comando Docker no estaba disponible. Se siguió la alternativa de servidores locales sin Docker.

7. Conclusiones

La ejecución permitió comprobar la conexión entre la orquestación de Prefect y el registro de experimentos de MLflow. El archivo lock conserva las versiones resueltas para facilitar la reproducción del entorno.

Como trabajo posterior quedan la definición del caso de uso, la preparación de datos y una pipeline propia de entrenamiento con métricas apropiadas.

8. Uso de herramientas de IA

Se utilizó Codex para orientar la configuración y proponer comandos.

La ejecución de comandos, la creación del repositorio y la operación de las interfaces fueron realizadas por mí, contando con asistencia de IA para aclarar conceptos o verificar flujo de trabajo.